



ALGORITMOS E PROGRAMAÇÃO COM IA

Pensamento computacional, Python e inteligência artificial para iniciantes

RESOLUÇÕES COMENTADAS DOS EXERCÍCIOS PROPOSTOS

Capítulo 3 — Quando Tudo Vira Bit

Como usar este material

Este material apresenta as resoluções comentadas dos exercícios propostos no Capítulo 3. Ele foi escrito para você, leitor ou leitora que está dando os primeiros passos em programação — por isso, cada resposta vem acompanhada de uma explicação do raciocínio envolvido, e não apenas do resultado final.

Recomendamos que você tente resolver cada exercício por conta própria antes de consultar a resolução. Se travar em algum ponto, leia apenas a parte *Como pensar antes de programar* e volte a tentar. Se você acertou, vale a pena ler a explicação mesmo assim: ela pode trazer um detalhe que ajuda a entender melhor o conceito.

Todos os códigos deste capítulo foram pensados para o Google Colab e podem ser copiados e executados diretamente. Sempre que possível, execute o código você mesmo e observe a saída antes de comparar com a saída esperada indicada aqui.

Cada exercício segue a mesma estrutura: o enunciado resumido, o raciocínio passo a passo, a resolução comentada, a saída esperada (quando houver código) e os erros mais comuns que costumam aparecer nesse tipo de exercício. Caixas adicionais — de atenção, dica ou autoavaliação — aparecem sempre que fazem sentido para aquele exercício.

Nota sobre unidades neste capítulo: todos os cálculos aqui adotam a convenção binária — 1 KB = 1.024 bytes e 1 MB = 1.024 × 1.024 bytes (1.048.576 bytes). É a convenção mais comum em contextos computacionais, e os Exercícios 5, 6, 7, 11, 12 e 13 dependem dela.

Visão geral dos exercícios

Use esta tabela para ter uma ideia geral do capítulo antes de começar. As resoluções detalhadas estão nas seções seguintes, organizadas pelo mesmo nível indicado aqui.

Exercício	Nível	Tema	O que você vai praticar
1	Aquecimento	Bit e byte	Diferenciar bit e byte e converter bytes em bits
2	Aquecimento	Potências de 2	Montar a tabela de pesos de um número binário de 8 bits

3	Aquecimento	Binário → decimal	Converter números binários para decimal
4	Aplicação	Decimal → binário	Converter números decimais para binário com divisões sucessivas
5	Aplicação	Imagem RGB	Calcular o tamanho de uma imagem Full HD sem compressão
6	Aplicação	Comparação de resoluções	Comparar o tamanho de imagens em diferentes resoluções
7	Aplicação	Áudio digital	Calcular o tamanho de um arquivo de áudio sem compressão
8	Análise e depuração	Depuração de cálculo	Encontrar um fator que faltou em um cálculo de imagem
9	Análise e depuração	Comentários no código	Reescrever comentários para explicar o porquê, não o quê
10	Análise e depuração	JPEG vs. PNG	Escolher o formato de imagem adequado para cada situação
11	Desafio	Notebook no Colab	Criar uma calculadora reutilizável de tamanho de imagem
12	Desafio	Notebook completo	Montar um notebook com Markdown, código e conclusão
13	Desafio	Escala e armazenamento	Estimar o armazenamento necessário para um grande volume de imagens
14	IA sob auditoria	Decimal → binário	Auditar uma explicação de IA sobre conversão para binário
15	IA sob auditoria	Código gerado por IA	Auditar um código de IA usando uma checklist de cinco critérios

Nível 1 — Aquecimento

Este capítulo mostra como imagens, sons e arquivos inteiros são, no fundo, apenas sequências muito longas de zeros e uns. Antes de calcular o tamanho de um arquivo, os três primeiros exercícios revisam — e aprofundam — a base que sustenta tudo: a diferença entre bit e byte, como as potências de 2 organizam o sistema binário, e como converter um número binário para decimal.

Exercício 1 — Bits e bytes

Nível: Aquecimento **Objetivo:** Diferenciar bit e byte e converter uma quantidade de bytes em bits.

Enunciado

Explique, com suas próprias palavras, a diferença entre bit e byte. Depois, responda: quantos bits existem em 5 bytes?

Como pensar antes de programar

Este exercício parece simples, mas confundir bit com byte é o erro de base que compromete todos os cálculos seguintes do capítulo — vale a pena fixar bem este conceito antes de seguir.

Passo	O que fazer / O que acontece
1	Um bit é a menor unidade de informação que um computador pode guardar. Ele só pode ter dois valores: 0 ou 1. Pense em um interruptor de luz: ligado (1) ou desligado (0).
2	Um byte é um grupo de 8 bits juntos — como uma palavra formada por 8 interruptores. Com 8 bits, existem 256 combinações possíveis (de 00000000 até 11111111).
3	Para encontrar quantos bits há em 5 bytes, basta multiplicar: 5 bytes $\times$ 8 bits por byte = 40 bits.

Resolução comentada

Resposta: **5 bytes = 40 bits.**

Por que 8 bits por byte?

A escolha de agrupar 8 bits em um byte não é arbitrária. Com 8 bits, é possível representar 256 valores diferentes (de 0 a 255) — um intervalo suficiente para representar letras, dígitos, símbolos e outros caracteres, como define a tabela ASCII que você viu no Capítulo 2. Por isso o byte se tornou a unidade padrão de armazenamento.

Erros comuns

- **Dizer que 1 byte = 1 bit:** são unidades diferentes — um byte contém 8 bits.
- **Multiplicar errado:** $5 \times 8 = 40$, não $5 + 8 = 13$. Bits e bytes se relacionam por multiplicação, não por soma.

Exercício 2 — Pesos binários

Nível: Aquecimento **Objetivo:** Montar a tabela de pesos de um número binário de 8 bits.

Enunciado

Complete a tabela de pesos para um número binário de 8 bits, da posição 2^7 até 2^0 .

Como pensar antes de programar

Cada posição em um número binário representa uma potência de 2. A posição mais à direita vale $2^0 = 1$. A cada casa para a esquerda, o peso dobra.

Resolução comentada

Posição	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Peso	128	64	32	16	8	4	2	1
Como calcular	$2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2$	$2 \times 2 \times 2 \times 2 \times 2 \times 2$	$2 \times 2 \times 2 \times 2 \times 2$	$2 \times 2 \times 2 \times 2$	$2 \times 2 \times 2$	2×2	2	1

Regra prática: comece pelo lado direito com 1. Para cada casa à esquerda, dobre o valor anterior: 1, 2, 4, 8, 16, 32, 64, 128.

Verifique no Python

Você pode verificar qualquer potência de 2 com `2**n`. Por exemplo, `print(2**7)` retorna 128 e `print(2**0)` retorna 1. Experimente no Colab com outros valores de `n`.

Erros comuns

- **Começar pelo lado errado:** $2^7 = 128$ fica à esquerda (posição mais significativa), e $2^0 = 1$ fica à direita (posição menos significativa).
- **Esquecer que $2^0 = 1$:** qualquer número elevado à potência zero é sempre 1 — inclusive 2. Isso costuma pegar quem está começando de surpresa.

Exercício 3 — Binário para decimal

Nível: Aquecimento **Objetivo:** Converter números binários para decimal somando os pesos das posições ativas.

Enunciado

Converta os números binários abaixo para decimal: a) 00000001 b) 00001010 c) 11111111

Como pensar antes de programar

Para converter binário em decimal, identifique as posições onde aparece o bit 1 e some os pesos correspondentes — usando a tabela de pesos do Exercício 2. O bit 0 não contribui com nada.

Resolução comentada

a) 00000001

Apenas a posição 2^0 está ativada (bit = 1). Todas as outras são zero.

$$00000001 = 1$$

b) 00001010

As posições 2^3 e 2^1 estão ativadas.

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Peso	128	64	32	16	8	4	2	1
Bit	0	0	0	0	1	0	1	0
Contribuição	0	0	0	0	8	0	2	0

$$00001010 = 8 + 2 = 10$$

c) 11111111

Todas as oito posições estão ativadas. Some todos os pesos:

$$11111111 = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$$

Por que 255 é um número importante na computação?

255 é o maior valor que cabe em 1 byte (8 bits, todos ativados). Esse valor aparece em muitos contextos: no modelo de cores RGB, que você viu no Capítulo 2, cada canal vai de 0 a 255; em endereços IP, cada parte também vai de 0 a 255; e em diversas tabelas de codificação, o byte é a unidade mínima. Quando você vê o número 255 em algum sistema computacional, quase sempre há um byte por trás dele.

Erros comuns

- **Somar as posições em vez dos pesos:** 00001010 tem bits nas posições 3 e 1 (contando da direita, a partir de 0) — mas os pesos dessas posições são 8 e 2, não 3 e 1.
- **Esquecer o alinhamento:** alinhe sempre o bit da direita com $2^0 = 1$. Zeros à esquerda não contribuem para a soma, mas ocupam posição na contagem.

Nível 2 — Aplicação

Agora você vai aplicar o sistema binário a problemas concretos: converter números decimais para binário usando o método das divisões sucessivas, e calcular o tamanho real de imagens e arquivos de áudio sem compressão. Todos os cálculos deste nível seguem a convenção indicada em *Como usar este material*: 1 KB = 1.024 bytes e 1 MB = 1.024 × 1.024 bytes.

Exercício 4 — Decimal para binário no papel

Nível: Aplicação **Objetivo:** Converter números decimais para binário pelo método das divisões sucessivas por 2.

Enunciado

Converta os números 13, 42, 7, 100 e 255 para binário usando divisões sucessivas por 2. Depois, verifique no Colab com `bin()`.

Como pensar antes de programar

O método das divisões sucessivas é o caminho inverso da soma de pesos que você praticou no Exercício 3.

Passo	O que fazer / O que acontece
1	Divida o número por 2. Anote o quociente e o resto (0 ou 1).
2	Repita o processo com o quociente, até ele chegar a 0.
3	Leia os restos de baixo para cima. Esse é o número em binário.

Resolução comentada

Exemplo detalhado com o número 13:

```

13 ÷ 2 = 6    -> resto 1
 6 ÷ 2 = 3    -> resto 0
 3 ÷ 2 = 1    -> resto 1
 1 ÷ 2 = 0    -> resto 1

Lendo de baixo para cima: 1101
Verificação: 8 + 4 + 0 + 1 = 13

```

Aplicando o mesmo processo aos demais números:

Decimal	Divisões sucessivas	Binário	Verificação
13	13→6(r1), 6→3(r0), 3→1(r1), 1→0(r1)	1101	8+4+1 = 13
42	42→21(r0), 21→10(r1), 10→5(r0), 5→2(r1), 2→1(r0), 1→0(r1)	101010	32+8+2 = 42
7	7→3(r1), 3→1(r1), 1→0(r1)	111	4+2+1 = 7
100	100→50(r0), 50→25(r0), 25→12(r1), 12→6(r0), 6→3(r0), 3→1(r1), 1→0(r1)	1100100	64+32+4 = 100
255	255→127(r1), 127→63(r1), ... , 1→0(r1) (oito divisões)	11111111	128+...+1 = 255

Verificação no Colab:

```

numeros = [13, 42, 7, 100, 255]

for n in numeros:
    print(f'{n:3d}    ->    {bin(n)}    ->    {n:08b}')

```

Saída esperada

```

13  ->  0b1101      ->  00001101
42  ->  0b101010    ->  00101010
7   ->  0b111       ->  00000111
100 ->  0b1100100   ->  01100100
255 ->  0b11111111  ->  11111111

```

O que o prefixo 0b significa?

O Python mostra `0b` na frente de todo número binário para indicar que a representação está em base 2. O `0b` não faz parte do número em si — é só uma etiqueta. Por isso `0b101010` e `101010` representam a mesma coisa: a forma binária do número 42.

Erros comuns

- **Ler os restos de cima para baixo:** o primeiro resto corresponde ao bit menos significativo (posição 2^0). Leia sempre de baixo para cima.
- **Parar antes do quociente chegar a zero:** o processo só termina quando o quociente for 0. Se parar antes, o número binário fica incompleto.
- **Cuidado com `bin()`:** ele retorna a string `'0b101010'`, e não o número 42 em si. Para usar esse valor em cálculos, use `int('101010', 2)` ou escreva `0b101010` diretamente no código.

Exercício 5 — Tamanho de imagem Full HD

Nível: Aplicação **Objetivo:** Calcular o tamanho bruto, em bytes e MB, de uma imagem RGB sem compressão.

Enunciado

Calcule o tamanho bruto, em bytes e em MB, de uma imagem RGB de 1920×1080 pixels sem compressão.

Como pensar antes de programar

Este cálculo combina três ideias: a quantidade total de pixels, o número de bytes que cada pixel ocupa (visto no Capítulo 2) e a conversão de bytes para KB e de KB para MB usando a base 1.024.

Passo	O que fazer / O que acontece
1 — Pixels	Calcule a quantidade total de pixels: largura $\times$ altura = $1920 \times 1080 = 2.073.600$ pixels.
2 — Bytes por pixel	Cada pixel RGB usa 3 bytes: 1 para vermelho (R), 1 para verde (G) e 1 para azul (B).
3 — Bytes totais	Bytes = pixels $\times$ 3 = $2.073.600 \times 3 = 6.220.800$ bytes.
4 — KB	KB = bytes $\div$ 1.024 = $6.220.800 \div 1.024 = 6.075$ KB.
5 — MB	MB = KB $\div$ 1.024 = $6.075 \div 1.024 \approx 5,93$ MB.

Resolução comentada

```

largura          = 1920
altura           = 1080
bytes_por_pixel  = 3          # R=1 byte, G=1 byte, B=1 byte

tamanho_bytes = largura * altura * bytes_por_pixel
tamanho_kb    = tamanho_bytes / 1024
tamanho_mb    = tamanho_kb / 1024

print(f'Pixels totais: {largura * altura:,}')
print(f'Tamanho: {tamanho_bytes:,} bytes')
print(f'Tamanho: {tamanho_kb:,.0f} KB')
print(f'Tamanho: {tamanho_mb:.2f} MB')
```

Saída esperada

```

Pixels totais: 2,073,600
Tamanho: 6,220,800 bytes
Tamanho: 6,075 KB
Tamanho: 5.93 MB
```

Por que 5,93 MB e não 6 MB?

Porque 1 MB não é exatamente 1.000.000 bytes. Em computação, $1 \text{ MB} = 1.024 \times 1.024 = 1.048.576$ bytes. Dividindo 6.220.800 por 1.048.576, obtemos 5,93. Se você usasse 1.000.000, chegaria a 6,22 MB — um resultado diferente. Por isso, sempre vale especificar qual convenção está sendo usada (este capítulo usa a binária, como explicado em Como usar este material).

Erros comuns

- **Esquecer o fator 3:** o erro mais frequente deste capítulo. 1920×1080 dá o número de pixels, não de bytes — cada pixel RGB precisa de 3 bytes.
- **Dividir por 1.000 em vez de 1.024:** o kilobyte computacional equivale a 1.024 bytes, não 1.000.
- **Não arredondar adequadamente:** 5,9296... arredonda para 5,93 MB, não para 6 MB.

Exercício 6 — Comparando resoluções

Nível: Aplicação **Objetivo:** Calcular e comparar o tamanho bruto de imagens em resoluções diferentes.

Enunciado

Calcule o tamanho bruto de duas imagens RGB sem compressão: a) 1280×720 ; b) 3840×2160 . Depois, responda: quantas vezes a imagem 4K é maior que a imagem 1280×720 ?

Como pensar antes de programar

A estrutura do cálculo é a mesma do Exercício 5 — apenas a resolução muda. Depois de calcular os dois tamanhos, a comparação é uma simples divisão entre eles.

Resolução comentada

a) Resolução HD (1280×720):

```
hd = 1280 * 720 * 3
print(f'HD 1280x720: {hd:,} bytes ({hd/1024/1024:.2f} MB)')
```

```
HD 1280x720: 2,764,800 bytes (2.64 MB)
```

b) Resolução 4K (3840×2160):

```
k4 = 3840 * 2160 * 3
print(f'4K 3840x2160: {k4:,} bytes ({k4/1024/1024:.2f} MB)')
```

```
4K 3840x2160: 24,883,200 bytes (23.73 MB)
```

Comparação: quantas vezes a 4K é maior que a 720p?

```
proporcao = k4 / hd
print(f'A imagem 4K é {proporcao:.0f}x maior que a 1280x720')
```

Saída esperada

```
A imagem 4K é 9x maior que a 1280x720
```

Por que exatamente 9 vezes?

A 4K tem o dobro da largura ($3840 = 2 \times 1920$) e o dobro da altura ($2160 = 2 \times 1080$) em relação à Full HD — dobrar largura e altura quadruplica a área ($2 \times 2 = 4$). Comparando a 4K diretamente com a 720p, a proporção é ainda maior: a 4K é 3 vezes mais larga e 3 vezes mais alta que a 720p, o que dá $3 \times 3 = 9$ vezes mais pixels. Como o número de bytes por pixel é o mesmo nas duas imagens, o arquivo também fica 9 vezes maior.

Exercício 7 — Áudio de uma entrevista

Nível: Aplicação **Objetivo:** Calcular o tamanho aproximado de um arquivo de áudio sem compressão.

Enunciado

Uma entrevista foi gravada em áudio estéreo, com 44.100 amostras por segundo e 16 bits por amostra. A duração é de 10 minutos. Calcule o tamanho aproximado do arquivo WAV sem compressão.

Como pensar antes de programar

Assim como nas imagens, o segredo é decompor o cálculo em etapas: quantos segundos de áudio existem, quantos bytes cada amostra ocupa, quantos canais são gravados ao mesmo tempo, e qual é o total.

Passo	O que fazer / O que acontece
1 — Duração	10 minutos = $10 \times 60 = 600$ segundos.
2 — Bytes por amostra	$16 \text{ bits por amostra} \div 8 \text{ bits por byte} = 2 \text{ bytes por amostra}$.
3 — Canais	Estéreo = 2 canais. Cada segundo de áudio precisa de $44.100 \times 2 \text{ bytes} \times 2 \text{ canais} = 176.400 \text{ bytes}$.
4 — Total	$176.400 \text{ bytes/s} \times 600 \text{ s} = 105.840.000 \text{ bytes}$.
5 — MB	$105.840.000 \div 1.048.576 \approx 100,94 \text{ MB}$.

Resolução comentada

```

freq_amostragem = 44100    # amostras por segundo
bits_por_amostra = 16      # profundidade de bits
canais           = 2        # estéreo
duracao_min      = 10       # minutos

duracao_seg      = duracao_min * 60
bytes_amostra    = bits_por_amostra // 8    # 16 bits = 2 bytes

tamanho_bytes    = freq_amostragem * bytes_amostra * canais * duracao_seg
tamanho_mb       = tamanho_bytes / 1_048_576

print(f'Duração: {duracao_seg} segundos')
print(f'Tamanho: {tamanho_bytes:,} bytes')
print(f'Tamanho: {tamanho_mb:.2f} MB')
```

Saída esperada

```

Duração: 600 segundos
Tamanho: 105,840,000 bytes
Tamanho: 100.94 MB
```

Por que // e não / para dividir bits por 8?

O operador // faz divisão inteira — descarta a parte decimal. Como $16 \text{ bits} \div 8 = 2$ é exato, o resultado é o mesmo que com /. Mas usar // é uma boa prática quando você sabe que o resultado deve ser um número inteiro de bytes, como neste caso. Se a

profundidade de bits fosse um número que não divide por 8 de forma exata, a divisão inteira ajudaria a perceber o problema.

Erros comuns

- **Esquecer de converter minutos para segundos:** a fórmula usa duração em segundos — 10 minutos = 600 segundos, não 10.
- **Ignorar o número de canais:** estéreo significa dois canais gravados simultaneamente. Cada canal dobra o volume de dados.
- **Não converter bits para bytes:** a fórmula usa bytes, não bits — 16 bits por amostra equivalem a 2 bytes por amostra.

Nível 3 — Análise e depuração

Os exercícios deste nível trazem um erro de cálculo, comentários de código que precisam de revisão e uma decisão técnica do dia a dia: escolher entre dois formatos de imagem. Em comum, todos pedem que você analise um código ou uma decisão já tomada por outra pessoa — uma habilidade tão importante quanto escrever código do zero.

Exercício 8 — Encontre o erro de unidade

Nível: Análise e depuração **Objetivo:** Identificar um fator que faltou em um cálculo de tamanho de imagem.

Enunciado

Um estudante calculou o tamanho de uma imagem RGB assim:

```
largura = 1920
altura  = 1080
tamanho = largura * altura
print(tamanho)
```

Explique o que está faltando no cálculo.

Como pensar antes de programar

Pergunte-se: o resultado deste código está em pixels ou em bytes? O enunciado pediu o tamanho da imagem — em qual unidade isso normalmente é medido?

Resolução comentada

O código calcula apenas a quantidade de pixels da imagem:

```
1920 x 1080 = 2.073.600 pixels
```

Mas o exercício pede o tamanho em bytes. Como você viu no Exercício 5, uma imagem RGB precisa de 3 bytes por pixel: 1 para o canal vermelho, 1 para o verde e 1 para o azul. Sem multiplicar por 3, o resultado está correto como contagem de pixels, mas errado como medida de tamanho de arquivo.

Código corrigido:

```
largura      = 1920
altura       = 1080
bytes_por_pixel = 3      # R=1 byte, G=1 byte, B=1 byte

tamanho = largura * altura * bytes_por_pixel
print(f'{tamanho:,} bytes')
```

Saída esperada

```
6,220,800 bytes
```

Como identificar esse tipo de erro rapidamente

Sempre pergunte: a unidade do resultado faz sentido? Se o código calcula `largura * altura` sem nenhum fator adicional, o resultado está em pixels, não em bytes. Para chegar a bytes em uma imagem RGB, o fator 3 é obrigatório. Para imagens com transparência (RGBA), o fator seria 4.

Exercício 9 — Comentário ruim, comentário bom

Nível: Análise e depuração **Objetivo:** Reescrever comentários de código para que expliquem a intenção, não o óbvio.

Enunciado

Reescreva os comentários abaixo para que expliquem melhor a intenção do código:

```
# largura é 1920
largura = 1920

# altura é 1080
altura = 1080

# multiplica largura por altura por 3
tamanho = largura * altura * 3
```

Como pensar antes de programar

Antes de reescrever, pergunte-se: o que cada comentário diz que o código ao lado já não diz por si só? Se a resposta for "nada", o comentário precisa de uma informação nova — geralmente sobre o porquê do valor, a unidade de medida, ou o raciocínio por trás da operação.

Resolução comentada

Os três comentários originais apenas repetem o que o código já mostra. Quem lê `largura = 1920` já sabe que `largura` é 1920. Um comentário que repete a linha não acrescenta nada. Bons comentários explicam por que aquele valor foi escolhido, qual é a unidade de medida, ou qual raciocínio está por trás da operação.

Versão corrigida:

```
# Largura padrão de uma imagem Full HD, em pixels
largura = 1920

# Altura padrão de uma imagem Full HD, em pixels
altura = 1080

# 3 bytes por pixel: 1 para vermelho, 1 para verde, 1 azul
# Resultado em bytes (sem compressão)
tamanho = largura * altura * 3
```

Comentário original	Problema	Comentário melhorado
# largura é 1920	Repete o código	# Largura de uma imagem Full HD, em pixels
# altura é 1080	Repete o código	# Altura de uma imagem Full HD, em pixels
# multiplica largura por altura por 3	Descreve a operação, não o motivo	# 3 bytes por pixel: R, G e B — resultado em bytes

Regra prática para comentários

Antes de escrever um comentário, pergunte: esse comentário diz algo que o código já não diz? Se a resposta for não, apague o comentário. Um código bem nomeado, com variáveis descritivas, quase não precisa de comentários nos trechos simples.

Comentários são mais úteis em cálculos menos óbvios, em conversões de unidade e em decisões que dependem de contexto externo ao código.

Exercício 10 — JPEG ou PNG?

Nível: Análise e depuração **Objetivo:** Escolher o formato de imagem adequado para diferentes situações.

Enunciado

Para cada situação, escolha JPEG ou PNG e justifique: a) foto de uma paisagem; b) captura de tela com texto; c) logotipo com fundo transparente; d) foto de perfil para rede social.

Como pensar antes de programar

A diferença fundamental entre JPEG e PNG está no tipo de compressão: o JPEG descarta alguns dados para reduzir o tamanho do arquivo (compressão com perda); o PNG reorganiza os dados sem descartar nada (compressão sem perda). Essa diferença é o que determina em que situação cada formato é mais adequado.

Resolução comentada

Situação	Formato	Justificativa
Foto de paisagem	JPEG	Fotografias têm muitas variações sutis de cor. O olho humano dificilmente percebe as pequenas perdas causadas pela compressão JPEG, e o arquivo fica muito menor.
Captura de tela com texto	PNG	Texto tem bordas nítidas e alto contraste. A compressão JPEG borra essas bordas, tornando o texto difícil de ler. O PNG preserva cada pixel exatamente.
Logotipo com fundo transparente	PNG	O JPEG não suporta transparência: um logotipo salvo em JPEG ficaria com fundo branco (ou outra cor sólida). Apenas formatos como PNG, WebP e GIF suportam canal alfa (transparência).
Foto de perfil para rede social	JPEG	É uma fotografia — a compressão JPEG produz um arquivo menor com qualidade aceitável para visualização em tela. As redes sociais geralmente recomprimem a imagem de qualquer forma ao fazer o upload.

Resumo da regra geral

Use JPEG para: fotografias, imagens com gradientes suaves de cor, e situações em que o tamanho do arquivo importa mais do que a perfeição dos detalhes.

Use PNG para: qualquer imagem com texto, bordas nítidas, logos, ícones, capturas de tela, imagens com fundo transparente, ou situações em que a fidelidade ao original é obrigatória.

Nível 4 — Desafio

Os exercícios 11, 12 e 13 pedem mais autonomia: construir uma calculadora reutilizável, montar um notebook completo com identidade própria e estimar o armazenamento necessário para um volume real de imagens — uma situação que aparece em decisões de negócio do dia a dia.

Exercício 11 — Calculadora de imagem no Colab

Nível: Desafio **Objetivo:** Construir um notebook que calcule o tamanho bruto de uma imagem RGB em bytes, KB e MB.

Enunciado

Crie um notebook no Colab que calcule o tamanho bruto de uma imagem RGB. O notebook deve permitir alterar largura e altura manualmente no código, e mostrar o resultado em bytes, KB e MB.

Como pensar antes de programar

Este exercício reúne tudo o que você praticou no Exercício 5, mas com um cuidado adicional: o código precisa ser fácil de reutilizar com outras resoluções, e fácil de entender por quem lê depois.

Passo	O que fazer / O que acontece
1 — Variáveis claras	largura e altura definidas no início, fáceis de alterar, com nomes que dizem o que são.
2 — Comentários úteis	Explicar que <code>bytes_por_pixel = 3</code> porque cada pixel RGB tem três canais de cor.
3 — Cálculo em etapas	Não fazer tudo em uma linha só — calcular bytes, depois KB, depois MB separadamente.
4 — Saída legível	Exibir o resultado com o nome da grandeza e a unidade, usando <code>{ : , }</code> para separador de milhar.
5 — Markdown	Pelo menos uma célula de título e uma de explicação do que o notebook faz.

Resolução comentada

```
# =====
# Calculadora de tamanho bruto de imagem RGB
# Altere largura e altura para testar outras
# resoluções
# =====

largura      = 1920    # pixels
altura       = 1080    # pixels
bytes_por_pixel = 3      # RGB: 1 byte por canal

# Cálculo
tamanho_bytes = largura * altura * bytes_por_pixel
tamanho_kb    = tamanho_bytes / 1024
tamanho_mb    = tamanho_kb    / 1024

# Resultado
print(f'Imagem:  {largura} x {altura} pixels')
```

```
print(f'Tamanho: {tamanho_bytes:,} bytes')  
print(f'Tamanho: {tamanho_kb:,.2f} KB')  
print(f'Tamanho: {tamanho_mb:,.2f} MB')
```

Saída esperada

```
Imagem: 1920 x 1080 pixels  
Tamanho: 6,220,800 bytes  
Tamanho: 6,075.00 KB  
Tamanho: 5.93 MB
```

Como testar se o seu código está correto

Use valores que você já conhece a resposta. Um ícone de 1×1 pixel deve retornar 3 bytes. Uma imagem de 10×10 pixels deve retornar 300 bytes. Se esses casos simples funcionarem, é provável que o código esteja certo também para resoluções maiores — essa técnica chama-se teste com casos conhecidos, e é útil sempre que você não tem certeza se um cálculo está correto.

Exercício 12 — Cartão de visita digital técnico

Nível: Desafio **Objetivo:** Montar um notebook completo, com Markdown, código e conclusão.

Enunciado

Crie um notebook chamado `Semana03_SeuNome.ipynb` com: (1) uma célula Markdown com título, nome, RA, curso e data; (2) uma seção "Sobre mim"; (3) uma seção "O que aprendi sobre bits"; (4) três células de código com cálculos de imagem, áudio ou conversão binária; (5) uma conclusão em Markdown.

Como pensar antes de programar

Este exercício não introduz nenhum cálculo novo — ele pede que você organize, em um único notebook, o que já foi praticado nos exercícios anteriores, com a identidade e a voz de quem o escreveu. Pense neste notebook como um pequeno cartão de visita técnico.

Resolução comentada (estrutura de referência)

Célula de texto (Markdown) — identificação:

```
# Ana Paula Souza

**RA:** 12345678
**Curso:** Ciência de Dados para Negócios — FATEC Votorantim
**Data:** 10/03/2026

---

## Sobre mim
Tenho interesse em *análise de dados* e *inteligência artificial*.

## O que aprendi sobre bits
Aprendi que computadores guardam tudo como 0 e 1.
Um bit vale 0 ou 1. Um byte vale 8 bits.
Uma imagem colorida é uma grade de pixels RGB.
```

Exemplo de célula de código que pode aparecer no notebook:

```
# Comparando o tamanho de três resoluções diferentes

resolucoes = [
    (640, 480, 'VGA'),
    (1920, 1080, 'Full HD'),
    (3840, 2160, '4K'),
]

for larg, alt, nome in resolucoes:
    tam = larg * alt * 3
    mb = tam / 1024 / 1024
    print(f'{nome:8s} {larg}x{alt} -> {tam:,} bytes ({mb:.1f} MB)')
```

Saída esperada

```
VGA      640x 480 -> 921,600 bytes (0.9 MB)
Full HD  1920x1080 -> 6,220,800 bytes (5.9 MB)
4K       3840x2160 -> 24,883,200 bytes (23.7 MB)
```

O que diferencia um notebook bom de um regular

Regular: células de código que funcionam, sem explicação do que estão fazendo.

Bom: células Markdown antes de cada bloco de código, explicando o problema; comentários no código explicando os fatores; resultado com unidades claras.

Excelente: além de tudo o que está acima, a conclusão em Markdown responde a três perguntas — o que aprendi? o que foi difícil? o que me surpreendeu?

Exercício 13 — Miniestudo de armazenamento

Nível: Desafio **Objetivo:** Estimar o armazenamento necessário para um grande volume de imagens, com e sem compressão.

Enunciado

Uma empresa precisa armazenar 50.000 imagens de produtos em resolução 1920×1080 RGB. Calcule o tamanho bruto sem compressão. Depois, estime quanto ocuparia se cada imagem JPEG tivesse, em média, 500 KB. Compare os dois resultados.

Como pensar antes de programar

A estratégia é calcular primeiro o tamanho de uma única imagem (como no Exercício 5), depois multiplicar pela quantidade de imagens — e repetir o processo para o cenário com compressão JPEG.

Resolução comentada

Cálculo 1 — sem compressão:

```
qtde_imagens    = 50_000
largura         = 1920
altura         = 1080
bytes_por_pixel = 3

bytes_por_imagem = largura * altura * bytes_por_pixel
total_bytes      = bytes_por_imagem * qtde_imagens
total_gb         = total_bytes / 1024**3

mb_unit = bytes_por_imagem / 1024 / 1024
print(f'Por imagem: {bytes_por_imagem:,} bytes ({mb_unit:.1f} MB)')
print(f'Total bruto: {total_bytes:,} bytes')
print(f'Total bruto: {total_gb:.2f} GB')

Por imagem: 6,220,800 bytes (5.9 MB)
Total bruto: 311,040,000,000 bytes
Total bruto: 289.68 GB
```

Cálculo 2 — com JPEG de 500 KB por imagem:

```
kb_por_imagem_jpeg = 500
total_kb_jpeg      = kb_por_imagem_jpeg * qtde_imagens
total_gb_jpeg      = total_kb_jpeg / 1024**2

print(f'Total JPEG: {total_kb_jpeg:,} KB')
print(f'Total JPEG: {total_gb_jpeg:.2f} GB')

reducao = total_bytes / (total_kb_jpeg * 1024)
print(f'Redução: {reducao:.1f}x menos espaço com JPEG')
```

Saída esperada

```
Total JPEG: 25,000,000 KB
Total JPEG: 23.84 GB
Redução: 12.2x menos espaço com JPEG
```

Sem compressão, o conjunto de imagens ocuparia quase 290 GB. Com JPEG de qualidade média, ocuparia cerca de 24 GB — uma redução de mais de 12 vezes. Em termos práticos, isso representa a diferença entre precisar de um servidor de armazenamento grande e caro, ou de um disco externo comum.

Conexão com decisões reais de negócio

Este tipo de cálculo aparece em projetos reais de dados. Antes de contratar armazenamento em nuvem, uma empresa precisa estimar quanto volume de dados terá. A diferença entre 290 GB e 24 GB pode representar centenas de reais por mês em custos de cloud. Saber fazer essa estimativa é uma habilidade concreta de Ciência de Dados para Negócios.

Atenção ao uso de 1024^2 e 1024^3

No código acima usamos 1024^{**2} para converter KB em GB ($1 \text{ GB} = 1.024 \times 1.024 \text{ KB}$) e 1024^{**3} para converter bytes em GB ($1 \text{ GB} = 1.024^3 \text{ bytes}$). Confira essa lógica: $1 \text{ GB} = 1.024 \text{ MB} = 1.024 \times 1.024 \text{ KB} = 1.024 \times 1.024 \times 1.024 \text{ bytes} = 1.073.741.824 \text{ bytes}$.

Nível 5 — IA sob auditoria

Como nos capítulos anteriores, este nível pede que você use uma IA, observe a resposta dela e a avalie criticamente — agora com foco em conversão decimal-binário e em código gerado para cálculos de imagem.

Exercício 14 — Peça explicação à IA e audite

Nível: IA sob auditoria **Objetivo:** Auditar uma explicação de IA sobre a conversão de 42 para binário.

Enunciado

Use o prompt abaixo para pedir a uma IA que explique a conversão de 42 para binário. Depois, responda às quatro perguntas de auditoria.

Explique para um estudante iniciante como converter o número decimal 42 para binário usando divisões sucessivas por 2. Mostre cada etapa, explique por que os restos são lidos de baixo para cima e inclua uma verificação somando potências de 2.

Como pensar antes de programar

Antes de auditar, é preciso saber o que você está buscando. A conversão de 42 para binário deve passar exatamente pelas seguintes etapas:

```
42 ÷ 2 = 21  -> resto 0
21 ÷ 2 = 10  -> resto 1
10 ÷ 2 =  5  -> resto 0
 5 ÷ 2 =  2  -> resto 1
 2 ÷ 2 =  1  -> resto 0
 1 ÷ 2 =  0  -> resto 1
```

Lendo de baixo para cima: 101010
Verificação: $32 + 8 + 2 = 42$

Resolução comentada

Confira a resposta da IA em relação às quatro perguntas de auditoria abaixo:

Pergunta	O que verificar	Resposta esperada
a) A explicação está correta?	Confira se as divisões batem com as mostradas acima. O resultado final deve ser 101010.	Sim, se mostrar todas as divisões e chegar a 101010.
b) A IA mostrou todas as etapas?	Conte as linhas de divisão — devem ser 6 (de 42 até o quociente chegar a 0).	Deve mostrar todas. Se apresentou só o resultado, a resposta está incompleta.
c) Explicou por que se lê de baixo para cima?	O primeiro resto corresponde ao bit de menor peso ($2^0 = 1$); os restos seguintes correspondem a pesos crescentes ($2^1, 2^2, \dots$).	Uma boa explicação menciona que o primeiro resto é o bit menos significativo.
d) O que você melhoraria?	Avalie se faltou a verificação por potências de 2, se os termos técnicos estão corretos, e se a	Incluir a verificação $32+8+2=42$ e uma tabela de pesos fortalece a resposta.

linguagem é acessível para iniciantes.	
--	--

Por que auditar uma resposta que parece certa?

Uma resposta de IA pode estar correta no resultado e errada na explicação — ou correta na explicação e incorreta em algum detalhe técnico. Programação exige precisão. Treinar o olhar crítico agora, com exemplos simples como este, é o que vai permitir que você use IA com segurança em projetos mais complexos.

Sinais de alerta em respostas de IA sobre binário

- **Resultado diferente de 101010:** a resposta está errada.
- **Resto lido de cima para baixo:** o processo foi invertido, dando 010101 em vez de 101010.
- **Verificação ausente:** respostas sem verificação por soma de potências são menos confiáveis para quem está começando.
- **Explicação sem os passos intermediários:** se a IA pulou direto para o resultado, a explicação não serve para quem está aprendendo o método.

Exercício 15 — Código gerado por IA

Nível: IA sob auditoria **Objetivo:** Auditar um código gerado por IA para calcular o tamanho de uma imagem RGB.

Enunciado

Peça a uma IA que gere um código Python para calcular o tamanho de uma imagem RGB. Em seguida, audite o código usando os cinco critérios abaixo.

Como pensar antes de programar

Abaixo está um exemplo de código adequado, gerado por IA. Use-o como ponto de comparação para o código que a sua IA gerar.

```
largura      = 1920
altura      = 1080
bytes_por_pixel = 3      # R=1 byte, G=1 byte, B=1 byte

tamanho_bytes = largura * altura * bytes_por_pixel
tamanho_kb    = tamanho_bytes / 1024
tamanho_mb    = tamanho_kb / 1024

print(f'Tamanho em bytes: {tamanho_bytes:,}')
print(f'Tamanho em KB:    {tamanho_kb:,.2f}')
print(f'Tamanho em MB:    {tamanho_mb:,.2f}')
```

Resolução comentada

Checklist de auditoria — aplicada ao código de referência acima:

Critério	O que verificar	O código acima passa?
1. Usa largura, altura e 3 bytes por pixel?	O fator 3 deve estar explícito, de preferência em uma variável com nome descritivo.	Sim — <code>bytes_por_pixel = 3</code> com comentário.
2. Converte corretamente para KB e MB?	Cada conversão divide por 1.024 (não por 1.000).	Sim — duas divisões encadeadas por 1024.
3. Usa nomes de variáveis compreensíveis?	Evitar nomes como <code>t</code> , <code>b</code> , <code>x</code> . Preferir <code>largura</code> , <code>tamanho_bytes</code> , <code>tamanho_mb</code> .	Sim — todos os nomes descrevem o que guardam.
4. Possui comentários úteis?	Pelo menos o comentário de <code>bytes_por_pixel</code> deve explicar os 3 canais de cor.	Sim — há comentário explicando R, G, B.
5. Você consegue explicar sem copiar?	Feche o código e explique em voz alta o que cada linha faz. Se travar, ainda não compreendeu.	Depende de você — esta coluna não pode ser preenchida por nós.

O que fazer se o código da IA não passar em algum critério

- **Faltou o fator 3:** peça à IA "Refaça o código incluindo explicitamente os 3 bytes por pixel do modelo RGB, com comentário explicando".
- **Dividiu por 1.000 em vez de 1.024:** peça à IA "Use a convenção binária, onde 1 KB = 1.024 bytes".
- **Nomes ruins, como `w`, `h`, `sz`:** peça à IA "Renomeie as variáveis para nomes que descrevam o que elas representam".

- **Você não consegue explicar:** volte ao raciocínio do Exercício 5 e releia a seção correspondente do capítulo. A IA não substitui o aprendizado neste caso.

Projeto???

Revisão do capítulo

Os quinze exercícios deste capítulo constroem uma progressão deliberada: você começou revisando bit, byte e o sistema binário, passou a calcular tamanhos reais de imagens e arquivos de áudio, analisou erros e comentários de código de outras pessoas, criou seus próprios notebooks e, por fim, avaliou criticamente explicações e códigos gerados por IA.

Se você teve dificuldade em algum exercício, o quadro abaixo indica o que vale revisar antes de seguir para o Capítulo 4:

Se você teve dificuldade em...	Vale revisar...
Diferenciar bit e byte	A seção sobre bits, bytes e a escala da informação, e o Exercício 1
Converter decimal para binário	A seção sobre o método das divisões sucessivas e o Exercício 4
Converter binário para decimal	A seção sobre o sistema binário (soma de pesos) e os Exercícios 2 e 3
Calcular o tamanho de uma imagem	A seção sobre cálculo de tamanho de imagem e os Exercícios 5, 6 e 8
Calcular o tamanho de um arquivo de áudio	A seção sobre cálculo de tamanho de áudio e o Exercício 7
Escolher entre JPEG e PNG	A seção sobre JPEG e PNG e o Exercício 10
Escrever comentários úteis no código	A seção sobre comentários no código e o Exercício 9
Organizar um notebook no Colab	A seção sobre como montar um notebook e os Exercícios 11 e 12
Auditar respostas ou códigos de IA	A seção sobre IA como apoio e os Exercícios 14 e 15

Você concluiu o terceiro capítulo do livro. A partir de agora, sempre que você ouvir falar em tamanho de arquivo, resolução de imagem ou qualidade de áudio, vale a pena lembrar: por trás de cada um desses números existe um cálculo simples — e, depois deste capítulo, você sabe refazê-lo.